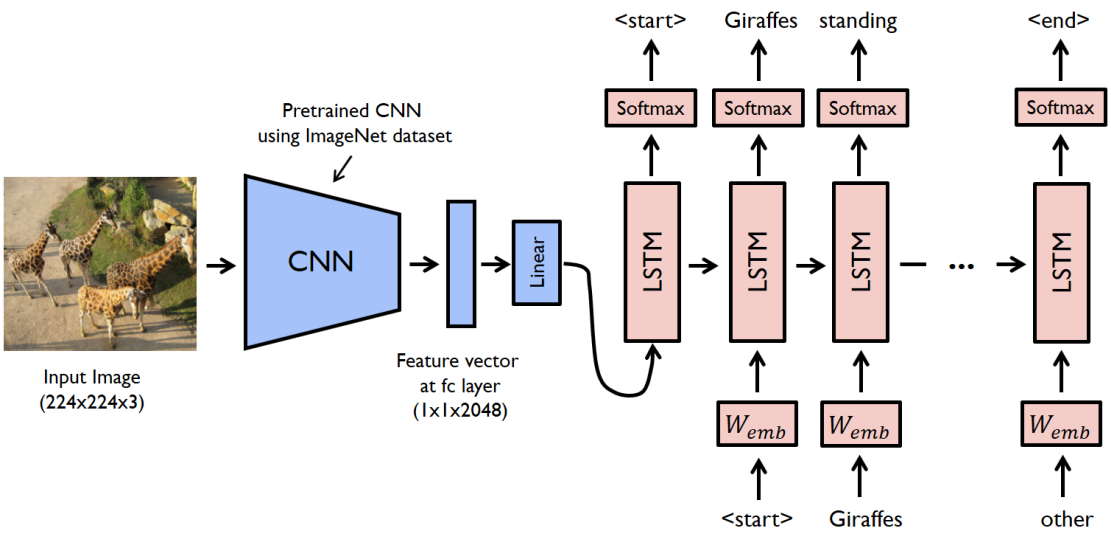
Tugas Besar 2 IF3270 Pembelajaran Mesin Convolutional Neural Network dan Recurrent Neural Network

Dipersiapkan Oleh Tim Asisten IF3270 2025/2026

Versi: 1.0 [20/04/2026]

Deadline: **Jumat, 15 Mei 2026**



Tujuan

Tugas Besar II pada kuliah IF3270 Pembelajaran Mesin bertujuan agar peserta kuliah mendapatkan wawasan tentang bagaimana cara mengimplementasikan Convolutional Neural Network (CNN) dan Recurrent Neural Network (RNN). Pada tugas ini, peserta kuliah akan ditugaskan untuk mengimplementasikan modul *forward propagation* CNN, Simple RNN, dan LSTM *from scratch*. Selain itu, peserta juga akan mengeksplorasi pipeline *image captioning* melalui arsitektur encoder-decoder yang menggabungkan CNN dan LSTM, yang merupakan fondasi dari banyak sistem deep learning modern seperti LLM multimodal dan speech recognition.

Referensi Pendukung

Keras & TensorFlow

- The Sequential model
- The Functional API (arsitektur encoder-decoder)
- Training & evaluation with Keras
- Save and load models (diperlukan untuk load bobot ke implementasi from scratch)
- Keras preprocessing layers
- tf.data pipeline guide

NumPy untuk Implementasi From Scratch

- NumPy broadcasting guide (untuk implementasi conv & matmul)
- numpy.einsum

CNN-RNN/LSTM Image Captioning Pipeline

- GeeksforGeeks — Image Caption Generator dengan Flickr8k + InceptionV3 + LSTM: tutorial end-to-end, arsitektur menggunakan init-inject (berbeda dengan spesifikasi tugas yang menggunakan pre-inject, namun masih sangat relevan)
- Nicola Fanelli — End-to-End Image Captioning dengan TensorFlow/Keras: penjelasan arsitektur *init-inject* secara detail
- Keras.io — Image Captioning (official example): referensi arsitektur encoder-decoder resmi dari Keras
- d2l.ai — RNN from Scratch: implementasi RNN dari nol dengan penanganan hidden state eksplisit
- d2l.ai — LSTM from Scratch: implementasi LSTM dari nol lengkap dengan bagian *Implementation from Scratch*
- d2l.ai — Sequence to Sequence (prediksi dengan initial state): menjelaskan greedy decoding step dengan initial state eksplisit

Konsep Arsitektur

- d2l.ai: CNN chapter
- d2l.ai: RNN chapter
- d2l.ai: LSTM
- d2l.ai: Deep RNN
- d2l.ai: Bidirectional RNN
- d2l.ai: Image Captioning (untuk bagian CNN-LSTM pipeline)
- Show and Tell: A Neural Image Caption Generator (Vinyals et al., 2015), paper arsitektur encoder-decoder yang digunakan
- Where to put the Image in an Image Caption Generator (Tanti et al., 2017), analisis perbandingan metode injection

Spesifikasi

Convolutional Neural Network

Bagian CNN dikerjakan dalam konteks task *image classification* menggunakan dataset Intel Image Classification (~25.000 gambar, 6 kategori: buildings, forest, glacier, mountain, sea, street), dengan split *train*, *validation*, dan *test* yang sudah tersedia.

Arsitektur Model dan Implementasi

Model CNN minimal harus memiliki jenis layer berikut (urutan dan jumlah layer dapat disesuaikan):

- Conv2D layer Implementasi *shared parameter* dan non-shared parameter
- Pooling layers
- Flatten/Global Pooling layer
- Dense layer

Loss function: Sparse Categorical Crossentropy. Optimizer: Adam.

Tahapan Pengerjaan

- **Bagian 1: Implementasi Utility Functions**
  Implementasikan utility functions berikut untuk pemrosesan data image menggunakan PIL/Pillow dan NumPy (tanpa Keras preprocessing):
  - **Image loader**: load gambar dari file path menggunakan PIL.Image.open, resize ke dimensi target, konversi ke numpy array, dan normalisasi pixel values ke rentang [0, 1]
  - **Batch loader**: load dan memproses sekumpulan gambar dari list file path menjadi numpy array dengan shape (N, H, W, C)
  - **Feature extractor**: menerima list path gambar, menggunakan Keras CNN encoder (frozen) untuk mengekstraksi feature vectors, dan menyimpan hasilnya ke disk (format .npy) agar tidak perlu diekstraksi ulang

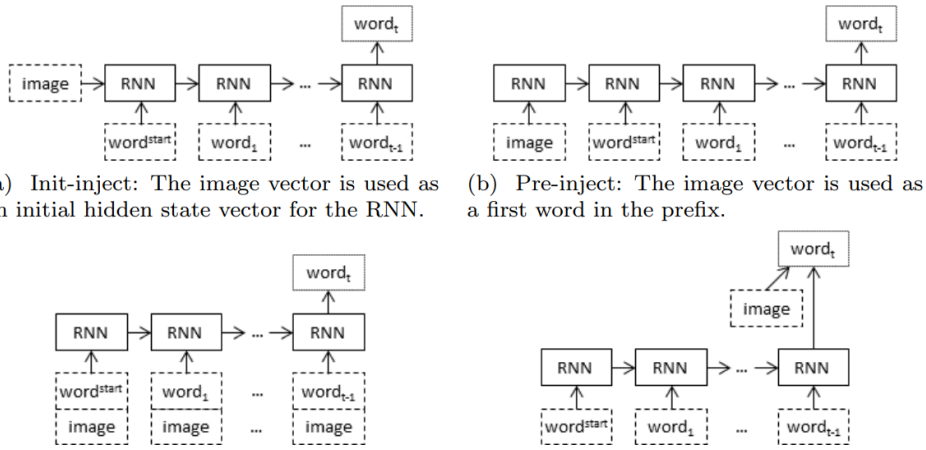
- **Bagian 2: Implementasi Forward Propagation From Scratch**
  - Implementasikan modul *forward propagation from scratch* yang dapat membaca bobot hasil pelatihan Keras. Implementasikan secara modular: setiap layer memiliki method *forward* tersendiri yang menerima input numpy array dan mengembalikan output numpy array.
  - Layer yang wajib diimplementasikan:
    - **Conv2D (shared parameters)**: operasi konvolusi 2D dengan sliding filter. Load bobot *kernel* (shape: [kH, kW, C_in, C_out]) dan *bias* dari Keras. Implementasikan operasi: untuk setiap posisi (i,j) dan filter k, hitung dot product antara patch input dan kernel[k], tambahkan bias[k], lalu aplikasikan fungsi aktivasi.
    - **LocallyConnected2D (non-shared parameters)**: konvolusi tanpa parameter sharing. Load bobot *kernel* (shape: [output_rows*output_cols, kH*kW*C_in, C_out]) dan *bias* dari Keras.
    - **MaxPooling2D / AveragePooling2D**: sliding window dengan operasi max atau average pada setiap channel.

Parameter: pool_size dan strides.
        - **GlobalAveragePooling2D / GlobalMaxPooling2D**: reduksi spasial seluruh feature map menjadi satu nilai per channel (average atau max).
        - **Flatten**: reshape tensor (H, W, C) menjadi vektor 1D dengan urutan row-major (C order), konsisten dengan Keras.
        - **Fungsi aktivasi**: implementasikan ReLU, Softmax, dan aktivasi lain yang digunakan dalam model.
    - Catatan: Khusus untuk **Dense layer**, Anda boleh menggunakan implementasi FFNN dari Tubes 1.
    - Referensi implementasi:
        - d2l.ai: Convolutions for Images (implementasi dari nol)
        - CS231n Notes: Convolutional Neural Networks

- **Bagian 3: Pelatihan Model**
    - Latih dua model CNN menggunakan Keras:
        - Model **Conv2D** (shared parameters): dijadikan baseline
        - Model **LocallyConnected2D** (non-shared parameters): pada posisi layer yang sama
    - Lakukan **variasi hyperparameter** berikut (hanya untuk model Conv2D / shared parameter):
        - Pengaruh **jumlah layer konvolusi**: pilih 3 variasi
        - Pengaruh **banyak filter per layer konvolusi**: pilih 3 variasi kombinasi
        - Pengaruh **ukuran filter per layer konvolusi**: pilih 3 variasi kombinasi
        - Pengaruh **jenis pooling layer**: pilih 2 variasi (max pooling atau average pooling)
    - Catatan: Gunakan macro F1-score sebagai metrik perbandingan.
    - Simpan bobot semua model hasil pelatihan.

- **Bagian 4: Eksperimen dan Evaluasi**
    - Jalankan forward propagation from scratch menggunakan **arsitektur terbaik** hasil pelatihan **Bagian 3**.
    - Lakukan analisis:
        - Bandingkan hasil *forward propagation from scratch* dengan Keras pada split data test menggunakan macro F1-score.
        - Untuk setiap variasi hyperparameter dari Bagian 3: bandingkan hasil akhir prediksi dan grafik *training/validation loss*, serta berikan kesimpulan pengaruh hyperparameter tersebut.
        - Untuk perbandingan shared vs non-shared: bandingkan macro F1-score, jumlah parameter, grafik *training/validation loss*, dan berikan kesimpulan bagaimana *parameter sharing* mempengaruhi kinerja dan efisiensi model.
        - Berikan kesimpulan dan analisis perbedaan hasil jika ada.

Simple Recurrent Neural Network dan Long Short-Term Memory Network



(a) Init-inject: The image vector is used as an initial hidden state vector for the RNN.

(b) Pre-inject: The image vector is used as a first word in the prefix.

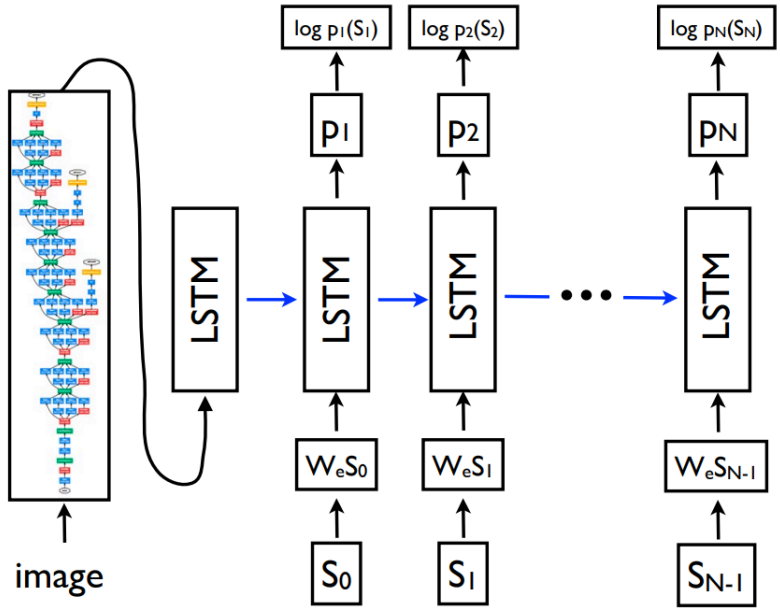
(c) Par-inject: The RNN accepts two inputs at once in every time step: a word and an image.

(d) Merge: The image vector is merged with the prefix outside of the RNN.

Gambar variasi arsitektur Image Captioning dari [paper (Tanti et.al., 2017)]

Bagian RNN dan LSTM dikerjakan dalam konteks task [image captioning] menggunakan dataset [Flickr8k] (8.092 gambar, 5 caption per gambar, split: 6.000 train / 1.000 validation / 1.000 test). Kedua arsitektur (RNN dan LSTM) dilatih secara terpisah sebagai decoder dan dibandingkan hasilnya.

Arsitektur Model



Gambar arsitektur dari [paper Show and Tell (Vinyals et al., 2015)]

Arsitektur mengacu pada Show and Tell (Vinyals et al., 2015), menggunakan *injection method pre-inject*: feature vector CNN di-project melalui Dense layer sebagai input x pada timestep t=-1, sebelum token **<start>**. Hidden state diinisialisasi dengan zeros (h_0) Untuk LSTM, $\mathbf{c_0}$ diinisialisasi zeros. Snippet dari paper (hal. 4):

$$
\begin{aligned}
x_{-1} &= \mathrm{CNN}(I) && (10)\\
x_t &= W_e S_t, \quad t \in \{0 \ldots N-1\} && (11)\\
p_{t+1} &= \mathrm{LSTM}(x_t), \quad t \in \{0 \ldots N-1\} && (12)
\end{aligned}
$$

Arsitektur model Keras untuk masing-masing decoder:

- Input layer untuk caption sequence yang sudah di-prepend dengan feature vector CNN (shape: (seq_len+1, embed_dim))
- Untuk caption words: Embedding layer: token → dense vector (embed_dim)
- Feature vector CNN di-project via Dense layer ke embed_dim, lalu di-concatenate di depan sequence embedding sebagai x_{-1}
- SimpleRNN atau LSTM dengan h_0 = zeros (default Keras)
- Dense output layer: → vocab_size, aktivasi softmax

Training menggunakan teacher forcing: untuk tiap gambar, sequence input adalah

[CNN_feature, emb(<start>), emb(S_0), ..., emb(S_{N-1})],

dengan target output adalah [S_0, S_1, ..., S_N] (digeser satu posisi).

Tahapan Pengerjaan

- **Bagian 0: Implementasi Forward Propagation from Scratch**
  - Implementasikan secara modular: setiap layer memiliki method *forward* tersendiri yang menerima dan mengembalikan numpy array.
  - Layer yang wajib diimplementasikan:
    - **Embedding layer**
    - **SimpleRNN cell**
    - **LSTM cell**
    - **Dense projection layer**: load bobot dan bias dari Keras. Forward pass: *output* = *W·x* + *b*, lalu project ke embed_dim untuk dijadikan x_{-1}.
    - **Dense output layer**: sama dengan Dense projection, dengan aktivasi softmax untuk menghasilkan distribusi probabilitas atas vocabulary.
  - Catatan: Khusus untuk **Dense layer**, Anda boleh menggunakan implementasi FFNN dari Tubes 1.
  - Referensi implementasi:
    - d2l.ai: RNN from Scratch
    - d2l.ai: LSTM from Scratch
    - Keras LSTM weights format (kernel, recurrent_kernel, bias)

- **Bagian 1: Feature Extraction (CNN Encoder, frozen):**
  - Gunakan **pretrained CNN** dari Keras (direkomendasikan InceptionV3 atau VGG16) tanpa classification head (*top layer*) dan dengan bobot ImageNet yang di-freeze.

- Lakukan *forward pass* seluruh gambar Flickr8k untuk **mengekstraksi *feature vectors*** dan **simpan** hasilnya ke disk (format .npy). Proses ini hanya dilakukan **sekali**, hasilnya digunakan bersama oleh decoder RNN dan LSTM.
- Gunakan **utility functions image** yang sudah dibuat pada bagian CNN untuk image loading.

- **Bagian 2: Preprocessing Caption:**
  - Lakukan tokenisasi caption dan bangun vocabulary dari seluruh caption training dengan special tokens **<start>**, **<end>**, **<pad>**. Gunakan fungsi-fungsi berikut (**tidak perlu** diimplementasikan from **scratch**):
    - tf.keras.layers.TextVectorization atau Python str.split(): tokenisasi caption (mapping kata → integer)
    - Python built-in str.lower(), re.sub(): lowercase dan hapus tanda baca
    - numpy.pad atau tf.keras.utils.pad_sequences: padding sequence ke panjang seragam
    - numpy.save / json.dump: menyimpan vocabulary dict ke disk
  - Referensi preprocessing caption:
    - TensorFlow Text: Tokenizers guide
    - NLTK tokenize: alternatif tokenizer berbasis Python
    - GeeksforGeeks: Caption preprocessing untuk image captioning (Flickr8k) (lihat bagian data preprocessing)

- **Bagian 3: Pelatihan Decoder menggunakan Keras (dilakukan dua kali: sekali untuk RNN, sekali untuk LSTM):**
  - Implementasikan arsitektur decoder sesuai deskripsi di atas. Loss function: Sparse Categorical Crossentropy. Optimizer: Adam.
  - Lakukan variasi pelatihan berikut untuk **masing-masing** decoder (**gunakan variasi yang sama untuk LSTM dan RNN**):
    - Pengaruh **jumlah layer recurrent**: pilih 3 variasi (misal 1, 2, 3 layer)
    - Pengaruh **ukuran hidden state**: pilih 2 variasi (misal 128, 512)
    - Minimal ada **6 variasi** per LSTM dan RNN **(total 12 variasi)**.
    - Nilai variasi bisa diubah, yang penting bisa menunjukkan pengaruhnya ke performa arsitektur.
  - Simpan bobot semua model hasil pelatihan.

- **Bagian 4: Implementasi Arsitektur (dilakukan untuk kedua decoder, RNN dan LSTM):**
  - Load bobot CNN encoder dari Keras pretrained (**Bagian 1**) dan bobot decoder dari hasil pelatihan **Bagian 3** (terpisah untuk RNN dan LSTM).
  - Implementasikan 2 arsitektur, 1 dengan RNN, 1 dengan LSTM, menggunakan komponen yang sudah dibuat from scratch, sesuai arsitektur di atas dari raw image hingga menghasilkan caption.

- **Bagian 5: Eksperimen**
  - Jalankan **pipeline** untuk setiap variasi pada **Bagian 3**. Kalkulasi dan catat BLEU-4 score dan waktu eksekusi untuk setiap variasi.

    - Pilih 1 **variasi arsitektur terbaik** untuk masing-masing LSTM dan RNN. Lalu, implementasi dan jalankan **arsitektur keras** yang setara, catat score dan waktu eksekusi.
    - Pilih 1 **arsitektur terbaik (**berdasarkan score dan waktu eksekusi**)** antara ke 4 variasi (LSTM vs RNN, Keras vs Scratch). Lalu, lakukan variasi **panjang maksimum caption** yang dihasilkan, **minimal 3 variasi**, catat score.

- **Evaluasi dan Analisis**
  a. **Variasi Jumlah Layer dan Hidden State**
    - Untuk setiap variasi jumlah layer dan ukuran hidden state dari **Bagian 3**:
      - Bandingkan BLEU-4 score dan METEOR score antara RNN dan LSTM pada split test Flickr8k
      - Bandingkan grafik training loss dan validation loss tiap epoch
    - Berikan kesimpulan bagaimana **jumlah layer** dan ukuran **hidden state** mempengaruhi kualitas caption dan waktu eksekusi.

  b. **Perbandingan Keras vs Scratch**
    - Membandingkan score dan waktu eksekusi Keras vs from scratch
    - Bandingkan grafik training loss dan validation loss tiap epoch
    - Berikan kesimpulan dan asumsi alasan perbedaan jika ada

  c. **Perbandingan RNN vs LSTM**
    - Membandingkan score dan waktu eksekusi antara decoder **RNN vs LSTM** (analisis utama)
    - Bandingkan grafik training loss dan validation loss tiap epoch
    - Lakukan **qualitative analysis**: tampilkan minimal 10 contoh gambar dengan **score yang bervariasi (tinggi, sedang, rendah)** beserta caption yang dihasilkan oleh decoder RNN dan LSTM, serta caption ground truth-nya. Identifikasi pola perbedaan antara keduanya.
    - Berikan analisis: **mengapa satu arsitektur (LSTM/RNN) menghasilkan caption yang lebih baik dibandingkan arsitektur lain**, kaitkan dengan konsep vanishing gradient dan kemampuan memori jangka panjang.

  d. **Pengaruh panjang maksimum caption**
    - Bandingkan pengaruhnya terhadap BLEU-4 score.
    - Beri kesimpulan.

Spesifikasi Bonus

Berikut merupakan beberapa spesifikasi bonus yang dapat Anda kerjakan:

- **[CNN] Visualisasi fitur**: implementasikan visualisasi intermediate feature maps dari layer konvolusi dan Grad-CAM untuk memvisualisasikan region gambar yang paling berpengaruh terhadap prediksi.
- **[RNN/LSTM] Image Captioning (init-inject)**: implementasikan arsitektur alternatif di mana image feature dan caption context digabungkan (misalnya melalui Add atau Concatenate layer) setelah RNN/LSTM selesai memproses caption prefix. Bandingkan hasil BLEU-4 dengan versi pre-inject. Paper referensi: [Paper].
- **[RNN/LSTM] Beam Search Decoder**: implementasikan beam search sebagai alternatif greedy decoding (k = 3 atau k = 5). Bandingkan kualitas caption yang dihasilkan.
- **[Semua] Batch inference**: seluruh implementasi *forward propagation from scratch* harus bisa menangani kasus batch inference, di mana model dapat menerima lebih dari satu input untuk satu kali forward propagation. Jumlah instance dalam satu batch dapat diatur dengan hyperparameter *batch_size*.
- **[Semua] Backward propagation**: implementasikan fungsi *backward propagation from scratch* untuk seluruh layer yang digunakan.

Kelompok

Pembagian kelompok ditentukan sendiri oleh mahasiswa dengan mengisi sheets kelompok IF3270 Pembelajaran Mesin dengan 1 kelompok terdiri dari **3 orang** dan **tidak lintas kelas**. Batas waktu pengisian kelompok adalah **Jumat, 24 April 2026**. Setelah waktu yang ditentukan, mahasiswa yang belum mengisi sheets kelompok akan diacak.

QnA

Pertanyaan dapat disampaikan pada **post Tugas Besar 2 di Teams**. Pastikan pertanyaan yang ditanyakan tidak berulang.

Aturan

Terdapat beberapa hal yang harus diperhatikan dalam pengerjaan tugas ini, yakni:

- Dilarang melakukan **plagiarisme, menggunakan AI secara tidak bertanggungjawab, dan melakukan kerjasama antar kelompok**. Pelanggaran pada poin ini akan menyebabkan pemberian **nilai E** pada setiap anggota kelompok.
- Tugas diimplementasikan dalam bahasa **Python**.
- Implementasi *forward propagation* dari CNN, RNN, dan LSTM *from scratch* hanya boleh menggunakan library untuk perhitungan matematika (contoh: NumPy, dll).
- Tidak ada batasan penggunaan library untuk segala sesuatu di luar implementasi *forward propagation*, *backward propagation*, dan data preprocessing CNN.

Deliverables

Tugas dikumpulkan dalam bentuk link ke repository GitHub yang minimal berisi beberapa hal berikut (**judul bab dan subab boleh diubah, selama maknanya sama**):

- Folder **src**, digunakan untuk menyimpan source code beserta dengan notebook pengujian.
- Folder **doc**, digunakan untuk menyimpan laporan dalam bentuk **.pdf** yang terdiri atas komponen berikut:
    - Cover
    - Deskripsi Persoalan
    - Pembahasan
    - Penjelasan implementasi (deskripsi kelas, atribut, dan method)
    - Penjelasan forward propagation
        - CNN
        - RNN
        - LSTM
    - Hasil pengujian
        - CNN
            - Perbandingan shared vs non-shared parameter
            - Pengaruh jumlah layer konvolusi
            - Pengaruh banyak filter per layer konvolusi
            - Pengaruh ukuran filter per layer konvolusi
            - Pengaruh jenis pooling layer
            - Perbandingan Keras vs from scratch
        - Simple RNN dan LSTM untuk Image Captioning
            - Perbandingan jumlah layer: RNN
            - Perbandingan jumlah layer: LSTM
            - Perbandingan ukuran hidden state: RNN
            - Perbandingan ukuran hidden state: LSTM
            - Perbandingan RNN vs LSTM
            - Perbandingan Keras vs from scratch
            - Pengaruh panjang maksimum caption terhadap BLEU-4
    - Kesimpulan dan Saran
    - Pembagian tugas tiap anggota kelompok
    - Referensi
    - Lampiran (Form Penggunaan AI)

- **README.md**, berisi deskripsi singkat repository, cara setup dan run program, dan pembagian tugas tiap anggota kelompok.
- Pengumpulan berupa **link github** dan dilakukan melalui **Edunex.**
- Batas akhir pengumpulan adalah **15 Mei 2026**.
- Pengumpulan dilakukan oleh **NIM terkecil.**

Referensi

- d2l.ai: LSTM
- d2l.ai: Deep RNN
- d2l.ai: Bidirectional RNN
- d2l.ai: RNN chapter
- d2l.ai: CNN chapter
- d2l.ai: Image Captioning
- d2l.ai: Beam Search
- numpy.einsum
- Vinyals et al. (2015): Show and Tell: A Neural Image Caption Generator
- Tanti et al. (2017): Where to put the Image in an Image Caption Generator
- Selvaraju et al. (2016): Grad-CAM
- Flickr8k dataset (Kaggle)
- Intel Image Classification dataset (Kaggle)
- Karpathy: The Unreasonable Effectiveness of RNNs
- CS231n Lecture 10: RNNs & Image Captioning (Stanford)